
Major Assignment 1: Hamming PUF

CS771: Introduction to Machine Learning

Kushagra Nayak

Roll Number: 240586
kushagra24@iitk.ac.in

Prathu Agarwal

Roll Number: 240782
prathuag24@iitk.ac.in

Abhishek Sardar

Roll Number: 240041
abhishekh24@iitk.ac.in

Pankaj Khoriya

Roll Number: 240729
kpankaj24@iitk.ac.in

1 Part 1: Mathematical Derivation

We want to find a feature map $\phi(\mathbf{c})$ and a linear model (w, b) such that

$$\frac{1 + \text{sign}(w^\top \phi(\mathbf{c}) + b)}{2} = r(\mathbf{c})$$

for every challenge $\mathbf{c} \in \{0, 1\}^{32}$. This is the standard linear classifier formulation covered in the course lectures [2]. The feature map uses cross-product terms to capture the quadratic structure of $h_e \cdot h_o$, following the PUF embedding idea from the same course material.

Step 1: Switch bits to ± 1

For each challenge bit c_i define $x_i = 1 - 2c_i \in \{-1, +1\}$. Similarly for each secret bit s_i define $w_i = 1 - 2s_i \in \{-1, +1\}$.

The XOR $m_i = c_i \oplus s_i$ can now be written as a product:

$$m_i = \frac{1 - x_i w_i}{2}.$$

This is easy to check: when $c_i = s_i$ we get $x_i w_i = 1$ so $m_i = 0$, and when $c_i \neq s_i$ we get $x_i w_i = -1$ so $m_i = 1$.

Step 2: Write h_e and h_o in terms of x_i

Let $A_e = \sum_{i \text{ even}} x_i w_i$ and $A_o = \sum_{j \text{ odd}} x_j w_j$. Then:

$$h_e = \sum_{i \text{ even}} m_i = \sum_{i \text{ even}} \frac{1 - x_i w_i}{2} = 8 - \frac{A_e}{2}, \quad (1)$$

$$h_o = \sum_{j \text{ odd}} m_j = \sum_{j \text{ odd}} \frac{1 - x_j w_j}{2} = 8 - \frac{A_o}{2}. \quad (2)$$

Step 3: Expand the product $h_e \cdot h_o$

$$h_e \cdot h_o = \left(8 - \frac{A_e}{2}\right) \left(8 - \frac{A_o}{2}\right) = 64 - 4A_e - 4A_o + \frac{A_e A_o}{4}. \quad (3)$$

Preprint.

Now expand $A_e A_o$:

$$A_e A_o = \left(\sum_{i \text{ even}} x_i w_i \right) \left(\sum_{j \text{ odd}} x_j w_j \right) = \sum_{\substack{i \text{ even} \\ j \text{ odd}}} w_i w_j (x_i x_j). \quad (4)$$

Putting (4) into (3):

$$h_e \cdot h_o = 64 - 4 \sum_{i \text{ even}} w_i x_i - 4 \sum_{j \text{ odd}} w_j x_j + \frac{1}{4} \sum_{\substack{i \text{ even} \\ j \text{ odd}}} w_i w_j (x_i x_j). \quad (5)$$

Step 4: Define the feature map and the linear model

Define the feature map (using only $\mathbf{c}$, not $\mathbf{s}$ or t):

$$\phi(\mathbf{c}) = \left[\underbrace{x_0, x_1, \dots, x_{31}}_{32 \text{ linear terms}}, \underbrace{x_0 x_1, x_0 x_3, \dots, x_{30} x_{31}}_{256 \text{ cross terms (even } i, \text{ odd } j)} \right] \in \mathbb{R}^{288}. \quad (6)$$

Now define the weight vector $w^* \in \mathbb{R}^{288}$ and bias $b^* \in \mathbb{R}$:

$$\begin{aligned} w_i^* &= -4 w_i \quad \forall i = 0, \dots, 31, \\ w_{[i,j]}^* &= \frac{1}{4} w_i w_j \quad i \text{ even, } j \text{ odd,} \\ b^* &= 64 - t. \end{aligned}$$

From (5), we get the key identity:

$$(w^*)^\top \phi(\mathbf{c}) + b^* = h_e \cdot h_o - t.$$

This is positive when $h_e h_o \geq t$ (response = 1) and negative when $h_e h_o < t$ (response = 0). So the **required result** is:

$$\frac{1 + \text{sign}((w^*)^\top \phi(\mathbf{c}) + b^*)}{2} = r(\mathbf{c})$$

for all $\mathbf{c} \in \{0, 1\}^{32}$ (with the convention $\text{sign}(0) = +1$ at the boundary $h_e h_o = t$, giving response 1).

Note: $\phi(\mathbf{c})$ uses only $\mathbf{c}$ (via $x_i = 1 - 2c_i$). The weights w^* and bias b^* depend on $\mathbf{s}$ and t , but these are learned by the SVM from CRP data.

2 Part 2: Dimensionality

Looking at equation (5), there are three types of terms:

1. The constant 64, which goes into the bias b .
2. Linear terms: $-4 \sum_{i \text{ even}} w_i x_i - 4 \sum_{j \text{ odd}} w_j x_j$. These need **all 32 features** $x_0, x_1, \dots, x_{31}$.
3. Cross-product terms: $\frac{1}{4} \sum_{i \text{ even}, j \text{ odd}} w_i w_j (x_i x_j)$. There are 16 even indices and 16 odd indices, giving **$16 \times 16 = 256$ cross-product features**.

The linear features cannot be removed. A term like x_{i_0} (for a fixed even index i_0) changes only with c_{i_0} , while each product $x_{i_0} x_j$ also changes with c_j . So no linear combination of cross-product features can replace x_{i_0} .

Therefore, the minimum dimensionality is:

$$\mathbf{D} = 32 + 256 = 288. \quad (7)$$

This is $O(n^2)$ in the bit-length $n = 32$, which is much smaller than an exponential map.

3 Part 4: Hyperparameter Experiments

All experiments use the public training set (7500 CRPs) and test set (2500 CRPs) with the feature map ϕ from equation (6). We report training time and test accuracy for LinearSVC and LogisticRegression from scikit-learn [1].

(a) Loss function in LinearSVC

Since LogisticRegression does not expose a loss hyperparameter, this experiment is specific to LinearSVC, run with $C = 0.9$, $\text{tol} = 10^{-4}$, and $\text{max_iter} = 5000$.

Table 1: Effect of loss on LinearSVC.

loss	dual	Train acc	Test acc	Time (s)
hinge	True	0.9988	0.9980	1.606
squared_hinge	False	1.0000	0.9980	0.170
squared_hinge	True	1.0000	0.9980	1.288

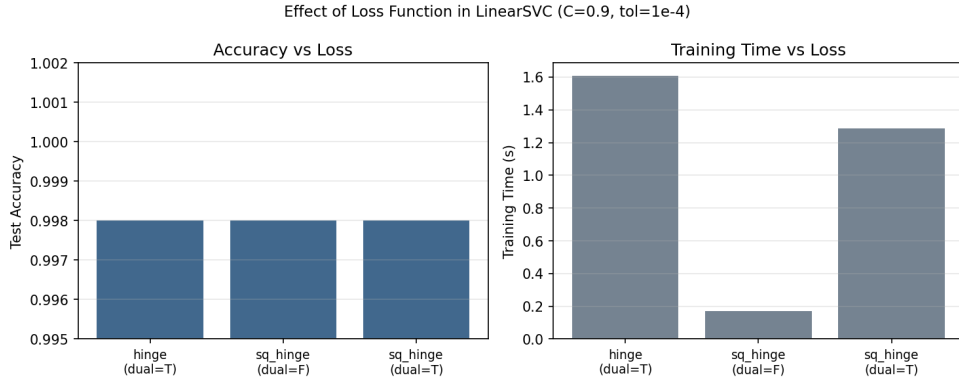


Figure 1: Effect of loss function on test accuracy (left) and training time (right).

Using squared_hinge with dual=False is **about 9× faster** than hinge with dual=True while achieving the same test accuracy. This is because the primal solver is more efficient when the number of samples (7500) greatly exceeds the number of features (288), as is the case here.

(b) Regularisation constant C

We varied C across six orders of magnitude with $\text{tol} = 10^{-4}$ and $\text{max_iter} = 10000$. LinearSVC used dual=False and LogisticRegression used the lbfgs solver.

Table 2: Effect of C on LinearSVC and LogisticRegression.

C	LinearSVC			LogisticRegression		
	Train	Test	Time (s)	Train	Test	Time (s)
0.001	0.9853	0.9680	0.125	0.9603	0.9396	0.018
0.01	0.9972	0.9876	0.134	0.9873	0.9736	0.016
0.1	0.9993	0.9980	0.148	0.9983	0.9920	0.020
0.9	1.0000	0.9980	0.167	0.9999	0.9960	0.026
10.0	1.0000	0.9976	0.497	1.0000	0.9968	0.028
100.0	1.0000	0.9972	0.837	1.0000	0.9972	0.026

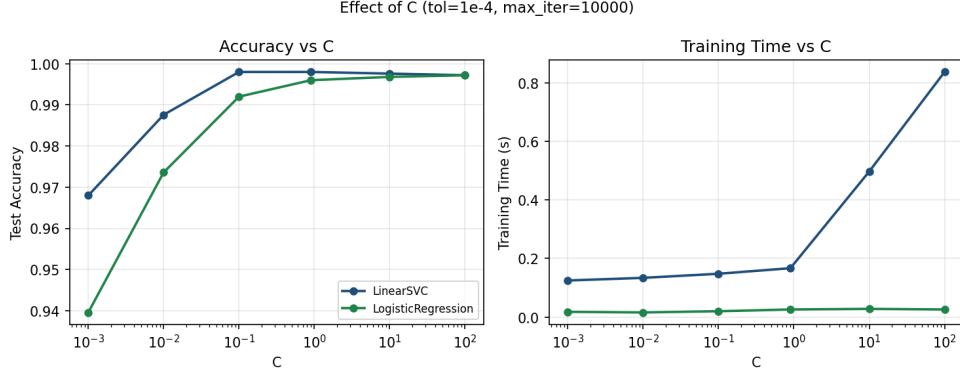


Figure 2: Effect of C on test accuracy (left) and training time (right) for both models.

Both models improve sharply up to $C \approx 0.1$ and then plateau, confirming the mapped data is nearly linearly separable. Pushing C to 100 only makes LinearSVC noticeably slower with no accuracy gain. $C = 0.9$ works well for both methods.

(c) Tolerance tol

Both models were run with $C = 0.9$ and $\text{max_iter} = 10000$ while tol was swept from 10^{-6} to 10^{-2} .

Table 3: Effect of tol on LinearSVC (dual=False) and LogisticRegression.

tol	LinearSVC			LogisticRegression		
	Train	Test	Time (s)	Train	Test	Time (s)
10 ⁻²	0.9995	0.9988	0.130	0.9925	0.9816	0.012
10 ⁻³	1.0000	0.9980	0.157	0.9996	0.9952	0.018
10 ⁻⁴	1.0000	0.9980	0.165	0.9999	0.9960	0.025
10 ⁻⁵	1.0000	0.9980	0.172	0.9999	0.9956	0.030
10 ⁻⁶	1.0000	0.9980	0.189	0.9999	0.9956	0.036

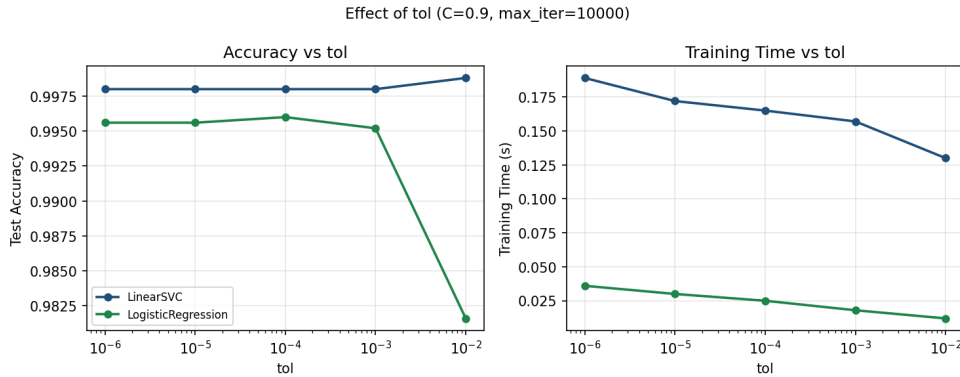


Figure 3: Effect of tol on test accuracy (left) and training time (right) for both models.

For LinearSVC, a looser tol (10^{-2}) actually gives slightly better test accuracy and trains faster, suggesting mild early stopping acts as regularisation. For LogisticRegression the opposite holds – at 10^{-2} the solver has not converged and accuracy drops to 98.16%. **We use $\text{tol} = 10^{-4}$ as a safe default for both.**

(d) Penalty: ℓ_1 vs ℓ_2

We compared ℓ_1 and ℓ_2 regularisation with $C = 0.9$, $\text{tol} = 10^{-4}$, and $\text{max_iter} = 5000$. For LogisticRegression, `lbfgs` was used with ℓ_2 and `liblinear` with ℓ_1 .

Table 4: Effect of penalty on LinearSVC and LogisticRegression.

Model	Penalty	Train acc	Test acc	Time (s)
LinearSVC	ℓ_2	1.0000	0.9980	0.172
LinearSVC	ℓ_1	1.0000	0.9988	4.140
LogisticRegression	ℓ_2	0.9999	0.9960	0.028
LogisticRegression	ℓ_1	1.0000	0.9956	1.674

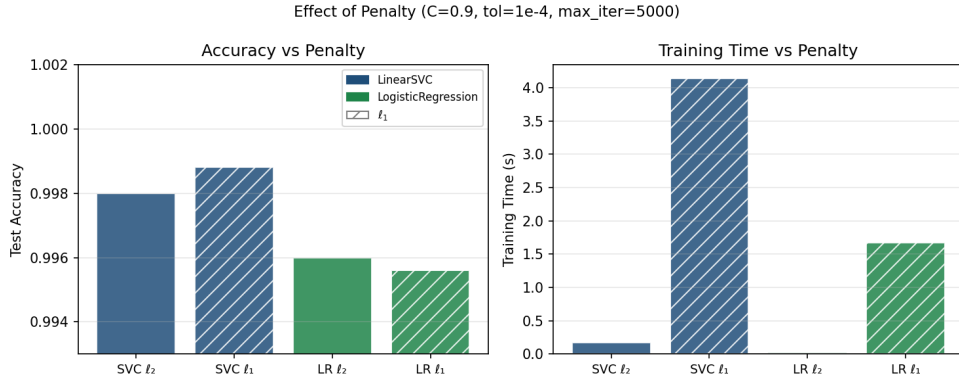


Figure 4: Effect of penalty on test accuracy (left) and training time (right). Solid bars = ℓ_2 , hatched bars = ℓ_1 .

ℓ_1 gives marginally better test accuracy for LinearSVC (99.88%) but is **about 24 $\times$ slower**. For LogisticRegression the two penalties produce identical test accuracy, yet ℓ_1 is far slower due to the coordinate-descent solver it requires. **Since the grading penalises training time, we use ℓ_2 .**

4 Part 5: Training Set Size vs Test Accuracy

No, 7500 CRPs are not all needed. Figure 5 shows how test accuracy changes as we vary the training set size. For each size, we ran 10 independent trials, each time picking a random subset (without replacement) from the original 7500-point training set. The test set (2500 CRPs) was kept fixed throughout.

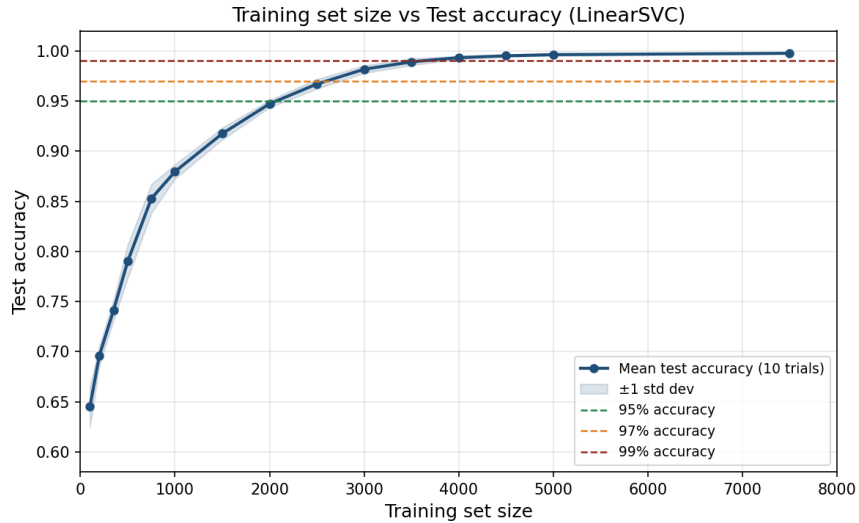


Figure 5: Test accuracy vs training set size (10 random trials per size). The shaded region shows ± 1 standard deviation. Dashed lines mark the 95%, 97%, and 99% accuracy thresholds.

Table 5 gives the numbers behind the plot.

Table 5: Test accuracy over 10 trials for each training set size.

Train size	Mean	Std	Min	Max
100	0.6450	0.0201	0.6172	0.6892
200	0.6959	0.0095	0.6860	0.7152
350	0.7412	0.0098	0.7312	0.7576
500	0.7901	0.0166	0.7660	0.8168
750	0.8527	0.0143	0.8368	0.8780
1000	0.8796	0.0070	0.8672	0.8896
1500	0.9175	0.0059	0.9032	0.9248
2000	0.9472	0.0036	0.9412	0.9524
2500	0.9670	0.0048	0.9556	0.9724
3000	0.9817	0.0037	0.9728	0.9868
3500	0.9891	0.0033	0.9840	0.9956
4000	0.9933	0.0015	0.9908	0.9956
4500	0.9951	0.0010	0.9936	0.9968
5000	0.9963	0.0012	0.9940	0.9980
7500	0.9976	0.0000	0.9976	0.9976

Minimum CRPs needed.

- $\geq 95\%$ test accuracy: around **2500 CRPs** (mean = 96.7%).
- $\geq 97\%$ test accuracy: around **3000 CRPs** (mean = 98.2%).
- $\geq 99\%$ test accuracy: around **4000 CRPs** (mean = 99.3%).

The accuracy rises quickly between 500 and 3500 training points, then slows down. This is expected because in the mapped space the data are nearly linearly separable, so once the model has seen enough examples it stops improving much.

References

- [1] Scikit-learn: LinearSVC and LogisticRegression documentation.
<https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>
- [2] Purushottam Kar. CS771: Introduction to Machine Learning, Course Materials. Indian Institute of Technology Kanpur, 2025–26.